

Atomicity and Transactions

Atomicity

In MongoDB, a write operation is atomic on the level of a single document, even if the operation modifies multiple embedded documents *within* a single document.

Multi-Document Transactions

When a single write operation (e.g. `db.collection.updateMany()`) modifies multiple documents, the modification of each document is atomic, but the operation as a whole is not atomic.

When performing multi-document write operations, whether through a single write operation or multiple write operations, other operations may interleave.

For situations that require atomicity of reads and writes to multiple documents (in a single or multiple collections), MongoDB supports distributed transactions, including transactions on replica sets and sharded clusters.

For more information, see [transactions](#)

IMPORTANT

In most cases, a distributed transaction incurs a greater performance cost over single document writes, and the availability of distributed transactions should not be a replacement for effective schema design. For many scenarios, the denormalized data model (embedded documents and arrays) will continue to be optimal for your data and use cases. That is, for many scenarios, modeling your data appropriately will minimize the need for distributed transactions.

For additional transactions usage considerations (such as runtime limit and oplog size limit), see also [Production Considerations](#).

Concurrency Control

Concurrency control allows multiple applications to run concurrently without causing data inconsistency or conflicts.

A `findAndModify` operation on a document is atomic: if the find condition matches a document, the update is performed on that document. Concurrent queries and additional updates on that document are not affected until the current update is complete.

Consider the following example:

- A collection with two documents:

```
db.myCollection.insertMany( [
```



```
{ _id: 0, a: 1, b: 1 },  
{ _id: 1, a: 1, b: 1 }  
]
```

- Two of the following `findAndModify` operations run concurrently:

```
db.myCollection.findAndModify( {  
  query: { a: 1 },  
  update: { $inc: { b: 1 }, $set: { a: 2 } }  
} )
```

After the `findAndModify` operations are complete, it is guaranteed that `a` and `b` in both documents are set to 2.

TIP

See also:

[findAndModify\(\) Upsert Example](#)

You can also create a unique index on a field so that it can only have unique values. This prevents inserts and updates from creating duplicate data. You can create a unique index on multiple fields to ensure the combination of field values is unique. For an example, see [findAndModify\(\) Upsert with Unique Index](#).

TIP

See also:

[Read Isolation, Consistency, and Recency](#)